

PROGRAMMING FOR PROBLEM SOLVING

String Functions

String Functions

S.No	Function	Description
1)	<code>strlen(string_name)</code>	returns the length of string name.
2)	<code>strcpy(destination, source)</code>	copies the contents of source string to destination string.
3)	<code>strcat(first_string, second_string)</code>	concatenates or joins first string with second string. The result of the string is stored in first string.
4)	<code>strcmp(first_string, second_string)</code>	compares the first string with second string. If both strings are same, it returns 0.
5)	<code>strrev(string)</code>	returns reverse string.
6)	<code>strlwr(string)</code>	returns string characters in lowercase.
7)	<code>strupr(string)</code>	returns string characters in uppercase.

strlen()

The strlen() function is used to find the length of a string. It returns the number of characters in a string, excluding the null terminator ('\0').

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Student";

    // Finding and printing length of string s
    printf("%lu", strlen(s));
    return 0;
}
```

strcpy()

The strcpy() function copies a string from the source to the destination. It copies the entire string, including the null terminator.

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char src[] = "Hello";
    char dest[20];

    // Copies "Hello" to dest
    strcpy(dest, src);
    printf("%s", dest);
    return 0;
}
```

Hello

strncpy()

```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello";
    char dest[20];

    // Copies "Hello" to dest
    strncpy(dest, src, 4);
    printf("%s", dest);
    return 0;
}
```

Hell

strcat()

The `strcat()` function is used to concatenate (append) one string to the end of another. It appends the source string to the destination string, replacing the null terminator of the destination with the source string's content.

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[30] = "Hello, ";
    char s2[] = "Students";
```

Hello, Students

```
// Appends "Students" to "Hello, "
strcat(s1, s2);
printf("%s", s1);
return 0; }
```

strncat()

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[30] = "Hello, ";
    char s2[] = "Students";

    // Appends "Students" to "Hello, "
    strncat(s1, s2, 4);
    printf("%s", s1);
    return 0;
}
```

Hello, Stud

strcmp()

The strcmp() is a built-in library function in C.

This function takes two strings as arguments, compares these two strings lexicographically and returns an integer value as a result of comparison.

strcmp()

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[] = "Apple";
    char s2[] = "Applet";

    // Compare two strings and print result
    int res = strcmp(s1, s2);
    if (res == 0)
        printf("s1 and s2 are same");
    else if (res < 0)
        printf("s1 is lexicographically smaller than s2");
    else
        printf("s1 is lexicographically greater than s2");
    return 0;
}
```

s1 is lexicographically smaller than s2

strncmp()

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s1[] = "Apple";
    char s2[] = "Applet";
```

```
    // Compare two strings upto 4 characters and print result
```

```
    int res = strncmp(s1, s2, 4);
```

```
    if (res == 0)
```

```
        printf("s1 and s2 are same");
```

```
    else if (res < 0)
```

```
        printf("s1 is lexicographically smaller than s2");
```

```
    else
```

```
        printf("s1 is lexicographically greater than s2");
```

```
    return 0;
```

```
}
```

s1 and s2 are same

strrev()

```
#include<stdio.h>
#include <string.h>
int main(){
    char str[20];
    printf("Enter string: ");
    gets(str); //reads string from console
    printf("String is: %s",str);
    printf("\nReverse String is: %s",strrev(str));
    return 0;
}
```

Enter string: student

String is: student

Reverse String is: tneduts

strlwr()

```
#include<stdio.h>
#include <string.h>
int main(){
    char str[20];
    printf("Enter string: ");
    gets(str);
    printf("String is: %s",str);
    printf("\nLower String is: %s",strlwr(str));
    return 0;
}
```

Enter string: STUdent

String is: STUdent

Lower String is: student

strupr()

```
#include<stdio.h>
#include <string.h>
int main(){
    char str[20];
    printf("Enter string: ");
    gets(str);
    printf("String is: %s",str);
    printf("\nUpper String is: %s",strupr(str));
    return 0;
}
```

Enter string: student

String is: student

Upper String is: STUDENT

strchr()

The `strchr()` function is used to locate the first occurrence of a character in a string. If the character is found, it returns a pointer to the first occurrence of the character; otherwise, it returns `NULL`.

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[] = "Hello, World!";

    // Finding the first occurrence of 'o' in string s
    char *res = strchr(s, 'o');

    if (res != NULL)
        printf("Character found at: %ld index", res - s);
    else
        printf("Character not found\n");
    return 0; }
```

Character found at: 4 index

strrchr()

In C, strrchr() function is similar to strchr() function used to find the last occurrence of a character in a string.

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Hello, World!";
    // Finding the last occurrence of 'o' in string s
    char *res = strrchr(s, 'o');

    if (res != NULL)
        printf("Character found at: %ld index", res - s);
    else
        printf("Character not found\n");
    return 0;
}
```

Character found at: 8 index

strstr()

The strstr() function in C is used to search the first occurrence of a substring in another string. If it is not found, it returns a NULL.

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s[] = "Hello, Students";
```

```
    // Find the occurrence of "Students" in string s
    char *pos = strstr(s, "Students");
```

```
    if (pos != NULL)
        printf("Found");
    else
        printf("Not Found");
    return 0;
```

```
}
```

Found: Students

strtok()

The strtok() function is used to split a string into tokens based on specified delimiters. It modifies the original string by replacing delimiters with null characters ('\0').

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[] = "Hello, Learn, C!";
    // Initializing tokens
    char *t = strtok(s, ", ");

    // Printing rest of the tokens
    while (t != NULL) {
        printf("%s\n", t);
        t = strtok(NULL, ", ");
    }
    return 0;
}
```

Token: Hello

Token: Learn

Token: C!

PROGRAMMING FOR PROBLEM SOLVING

Array of Strings

Array of Strings

- In C, an array of strings is a 2D array where each row contains a sequence of characters terminated by a '\0' NULL character (strings).
- It is used to store multiple strings in a single array.

Syntax of Array of Strings

```
char arr_name[r][m] = {s1, s2, ..., sn};
```

Here,

- arr_name: Name of the variable.
- r: Maximum number of strings to be stored in the array
- m: Maximum number of character values that can be stored in each string.
- s1, s2, ..., sn: Strings to be stored.

example

```
#include <stdio.h>
```

```
int main() {
```

```
    // Creating array of strings for 3 strings with max length of each string as 10
```

```
    char arr[3][10] = {"memo", "memory", "memoryfor"};
```

```
    int i;
```

```
    for (i = 0; i < 3; i++)
```

```
        printf("%s\n", arr[i]);
```

```
    return 0;
```

```
}
```

memo

memory

memoryfor

Invalid Operations in Arrays of Strings

can't directly change or assign the values to an array of strings in C.

Example:

```
char arr[3][10] = {"memo", "memory", "memoryfor"};  
arr[0] = "GFG";
```

// This will give an Error that says assignment to expression with an array type.

To change values we can use strcpy() function in C:

```
strcpy(arr[0], "GFG");
```

// This will copy the value to the arr[0].

How Array of Strings are Represented in Memory?

Consider the array of string:

```
char arr[3][10] = {"memo", "memory", "memoryfor"};
```

It will be stored in the format shown below:

	0	1	2	3	4	5	6	7	8	9
arr[0]	m	e	m	o	\0					
arr[1]	m	e	m	o	r	y	\0			
arr[2]	m	e	m	o	r	y	f	o	r	\0

The diagram illustrates the memory layout of the array `arr`. It shows three rows representing the strings "memo", "memory", and "memoryfor". Each row has 10 columns, indexed 0 to 9. The first five columns (0-4) contain the characters of the string, and the sixth column (5) contains the null terminator `\0`. The remaining columns (6-9) are shaded gray and labeled "memory wastage". A black bracket highlights the null terminator in the first row, and another black bracket highlights the null terminator in the second row.

Array of Strings Using Pointers

In C we can use an Array of pointers instead of having a 2-Dimensional character array. It is a single-dimensional array of Pointers where pointer to the first character of the string literal is stored.

Syntax:

```
char *arr[] = {"memo", "memory", "memoryfor"};
```

char *: It indicates that each element in the array is a pointer to a character (i.e., each points to a string).

arr[]: Declares an array of pointers, where the size is determined by the number of strings in the initialization list.

= {"memo", "memory", "memoryfor"}: Initializes the array, with each pointer pointing to a string: arr[0] points to "memo", arr[1] to "memory", and arr[2] to "memoryfor".

example

```
#include <stdio.h>
```

```
int main() {
```

```
    // Array of pointers to strings
```

```
    char *arr[] = {"memo", "memory", "memoryfor"};
```

```
    for (int i = 0; i < 3; i++)
```

```
        printf("%s\n", arr[i]);
```

```
    return 0;
```

```
}
```

memo

memory

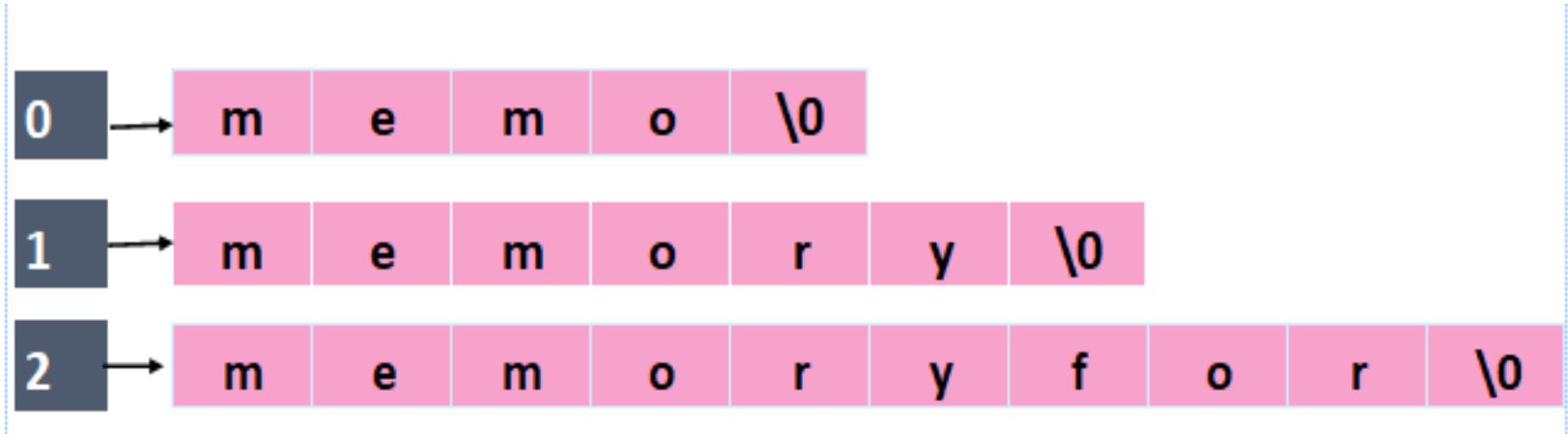
memoryfor

How Array of String Pointers Represented in Memory?

Unlike the 2D array of characters, array of pointer to strings store the pointer to the array. So, each string is only take the required amount of memory.

```
char *arr[] = {"memo", "memory", "memoryfor"};
```

It will be stored in the format shown below:



Sorting Array of names

```
#include <stdio.h>
#include <string.h>
int main() {
    int n;
    printf("Enter number of names: ");
    scanf("%d", &n);
    char names[100][100];
    printf("Enter %d names:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%s", names[i]);
    } // Sorting logic (Bubble Sort)
    char temp[100];
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (strcmp(names[j], names[j+1]) > 0) {
```

```
                strcpy(temp, names[j]);
                strcpy(names[j], names[j+1]);
                strcpy(names[j+1], temp);
            }
        }

        printf("\nSorted names:\n");
        for (int i = 0; i < n; i++) {
            printf("%s\n", names[i]);
        }

        return 0;
    }
```

Sorting Array of names

OUTPUT:

Enter number of names: 3

Enter 3 names:

practice

pps

programs

Sorted names:

pps

practice

programs

Sorting Array of names

```
#include <stdio.h>
#include <string.h>
int main() {
    int n;
    printf("Enter number of names: ");
    scanf("%d", &n);
    char names[100][100];
    char *ptr[100];
    printf("Enter %d names:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%s", names[i]);
        ptr[i] = names[i];
    } // Sorting logic (Bubble Sort)
    char *temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
```

```
            if (strcmp(names[j], names[j+1]) > 0) {
                temp = ptr[j];
                ptr[j] = ptr[j+1];
                ptr[j+1] = temp;
            }
        }
    }

    printf("\nSorted names:\n");
    for (int i = 0; i < n; i++) {
        printf("%s\n", ptr[i]);
    }

    return 0;
}
```

ASSIGNMENT AND SUBSTRINGS

Invalid string assignment:

```
char s[10];
```

```
s = "Hello"; // ERROR
```

Valid assignment:

```
char s[10];
```

```
strcpy(s, "Hello");
```

Substring Extraction

```
char s[] = "HelloWorld";  
char sub[10];
```

```
strncpy(sub, s + 5, 5); // copies "World"  
sub[5] = '\0';
```

```
printf("%s", sub);
```

CHARACTER POINTERS

Pointer to string literal (read-only)

```
char *p = "Hello";
```

- Stored in read-only memory
- You cannot modify it:

```
p[0] = 'J'; // ERROR
```

Pointer to character array (modifiable)

```
char s[] = "Hello";
```

```
char *p = s;
```

```
p[0] = 'J'; // Works
```

```
printf("%s", s); // Output: Jello
```

FUNCTIONS WITH CHARACTER POINTERS

Example: function to count length:

```
int length(char *p) {  
    int i = 0;  
    while (p[i] != '\0')  
        i++;  
    return i;  
}
```

```
int main() {  
    printf("%d", length("Hello"));  
}
```